LAPORAN

UTS PEMBANGUNAN PERANGKAT LUNAK BERORIENTASI SERVICE

Laporan ini disusun untuk memenuhi salah satu penilaian tugas mata kuliah Pembangunan Perangkat Lunak Berorientasi Service

Dosen Pengampu:

Muhammad Panji Muslim, S.Pd., M.Kom

Disusun Oleh:

Praffi Ramadhani 2410511028

PROGRAM STUDI S1 INFORMATIKA FAKULTAS

ILMU KOMPUTER

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAKARTA

2026

BAB 1 – PENDAHULUAN

1.1 Latar Belakang

Sistem manajemen kos merupakan salah satu aplikasi yang memiliki kompleksitas tinggi karena melibatkan berbagai domain bisnis yang berbeda, seperti manajemen properti, autentikasi pengguna, dan proses pemesanan serta pembayaran. Pendekatan monolith tradisional memiliki keterbatasan dalam hal skalabilitas dan pemeliharaan ketika sistem berkembang.

Arsitektur microservice menjadi solusi yang tepat untuk mengatasi permasalahan tersebut. Dengan memisahkan setiap domain bisnis menjadi service yang independen, sistem dapat dikembangkan, di-deploy, dan di-scale secara terpisah sesuai kebutuhan.

1. 2 Tujuan

Membangun sistem REST API manajemen kos berbasis arsitektur microservice yang terdiri dari tiga service independen dan satu API Gateway, dengan fitur autentikasi JWT, Google OAuth 2.0, dan rate limiting.

1.3 Ruang Lingkup

Sistem mencakup:

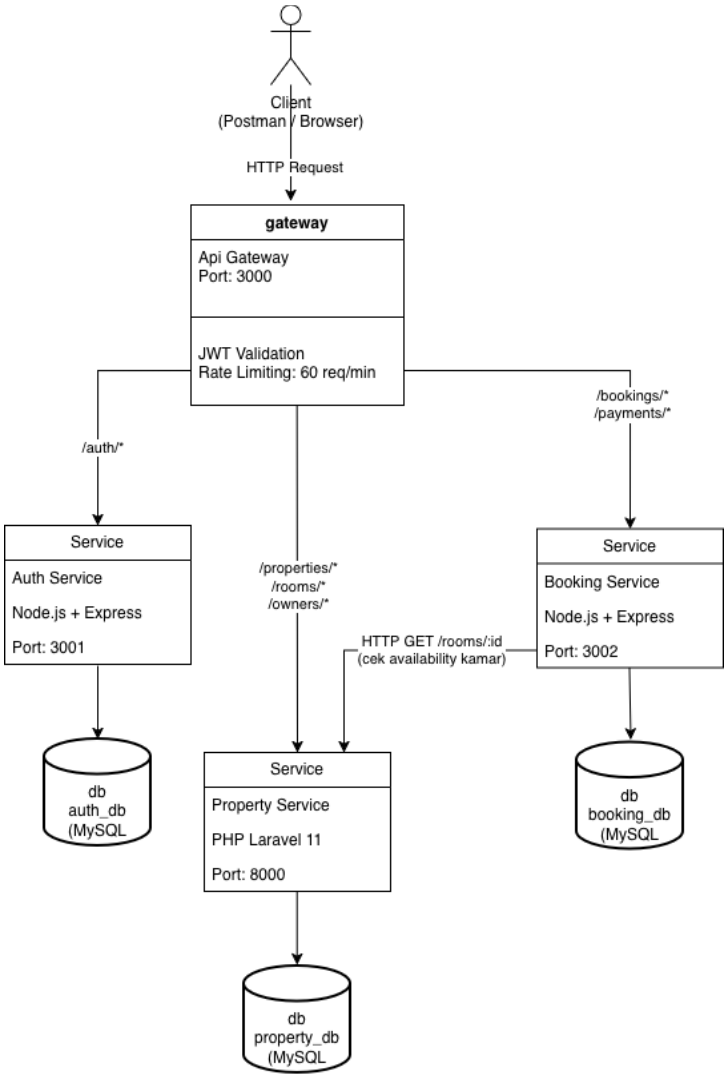
- Manajemen akun pengguna (autentikasi dan otorisasi)
- Manajemen properti, kamar, dan pemilik kos
- Pemesanan kamar dan pencatatan pembayaran
- API Gateway sebagai single entry point

BAB 2 – ARSITEKTUR SISTEM

2.1 Ruang Lingkup

Sistem terdiri dari 3 microservice independen + 1 API Gateway. Setiap service memiliki database MySQL tersendiri dan tidak berbagi tabel secara langsung antar service.

2.2 Arsitektur Sistem



Gambar di atas menunjukkan arsitektur sistem manajemen kos secara keseluruhan. Seluruh request dari client (Postman atau browser) masuk melalui API Gateway pada port 3000. Gateway bertanggung jawab melakukan validasi JWT dan rate limiting sebelum meneruskan request ke service yang sesuai berdasarkan path prefix.

Terdapat tiga service di balik gateway, masing-masing dengan database MySQL tersendiri. Auth Service menangani autentikasi di port 3001. Property Service berbasis Laravel mengelola data properti dan kamar di port 8000. Booking Service menangani pemesanan dan pembayaran di port 3002.

Tanda panah dari Booking Service ke Property Service menggambarkan komunikasi antar service ketika user membuat booking. Booking Service melakukan HTTP call ke Property Service untuk memverifikasi ketersediaan kamar sebelum booking dikonfirmasi.

2.3 Komponen Sistem

Service	Tech Stack	Port	Database
API Gateway	Node.js + Express	3000	-
Auth Service	Node.js + Express	3001	auth_db (MySQL)
Property Service	PHP Laravel 11	8000	property_db (MySQL)
Booking Service	Node.js + Express	3002	Booking_db (Mysql)

2.4 Deskripsi Tiap Service

- API Gateway
  Berperan sebagai single entry point untuk semua request dari client. Melakukan validasi JWT sebelum meneruskan request ke service yang sesuai, serta menerapkan rate limiting 60 request per menit per IP menggunakan library express-rate-limit.
- Auth Service
  Mengelola autentikasi pengguna dengan JWT (access token 15 menit + refresh token 7 hari). Mendukung login standar (email/password) dan Google OAuth 2.0 Authorization Code Flow. Menyimpan nama, email, foto profil dari Google dan menandai akun dengan flag oauth_provider.
- Property Service
  Dibangun dengan PHP Laravel 11 mengikuti pola MVC. Mengelola data owners, properties, rooms, dan facilities dengan relasi Eloquent. Mendukung pagination dan filtering pada endpoint listing.
- Booking Service
  Mengelola proses pemesanan kamar dan pencatatan pembayaran. Melakukan HTTP call ke property-service untuk memverifikasi ketersediaan kamar sebelum membuat booking.

2.5 Inter-Service Communication

Booking Service berkomunikasi dengan Property Service melalui HTTP call internal:

- POST /bookings
  - ⇒ Booking Service
    - ⇒ HTTP GET /rooms/:id ke Property Service
    - ⇒ Verifikasi kamar tersedia
    - ⇒ Buat data booking

BAB 3 – JUSTIFIKASI ARSITEKTUR

3.1 Perbandingan Microservice vs Monolitik

Aspek	Monolitik	Microservice
Deployment	Seluruh aplikasi harus di-redeploy meskipun hanya satu fitur yang berubah	Setiap service di-deploy independen, update auth tidak mengganggu booking
Skalabilitas	Seluruh aplikasi harus di-scale meskipun hanya satu bagian yang padat traffic	Hanya service yang membutuhkan yang di-scale
Tech Stack	Satu bahasa/framework untuk semua fitur	Setiap service bisa menggunakan stack yang paling sesuai, property pakai Laravel, yang lain node.js
Fault Isolation	Jika satu fitur crash, seluruh aplikasi bisa ikut down	Jika booking service down, user masih bisa browsing properti
Database	Semua fitur berbagi satu databse, perubahan skema berisiko merusak fitur lain	Setiap service punya databse sendiri, perubahan terisolasi

Kompleksitas awal	Lebih simple, satu codebase, satu deployment	Lebih kompleks, perlu gateway, Docker, inter-service communication

3.2 Mengapa API Gateway

- Single Entry Point: Client hanya perlu tahu satu URL, bukan URL tiap service
- JWT Validation Terpusat: Validasi token dilakukan di gateway, tidak perlu duplikasi logika di setiap service
- Rate Limiting Terpusat: Perlindungan dari abuse request dilakukan di satu tempat
- Routing Transparan: Client tidak perlu tahu service mana yang menangani request

3.3 Mengapa Laravel untuk Property Service

- Requirement soal: Property service wajib menggunakan PHP dengan pola MVC
- Eloquent ORM: Memudahkan pengelolaan relasi antar 4 tabel (owners, properties, rooms, facilities)
- Form Request: Validasi input terpisah dari controller, kode lebih bersih
- Migration: Pengelolaan skema database yang terstruktur dan mudah di-rollback

3.4 Mengapa Node.js untuk Auth dan Booking Service

- Non-blocking I/O: Cocok untuk service yang banyak melakukan operasi database dan HTTP call
- Ekosistem JWT: Library seperti jsonwebtoken dan passport sudah matang
- Konsistensi: Gateway, auth, dan booking service dalam satu ekosistem memudahkan debugging

3.5 Database Isolation

Setiap service memiliki database terpisah (auth_db, property_db, booking_db). Ini memastikan:

- Tidak ada ketergantungan langsung antar service melalui tabel database
- Setiap service bisa menggunakan skema yang paling sesuai
- Perubahan skema di satu service tidak mempengaruhi service lain

BAB 4 – ROUTING MAP

Semua request dari client masuk melalui API Gateway (port 3000) yang kemudian meneruskan ke service yang sesuai.

4.1 Auth Service (/auth/*)

Method	Path	Akses	Keterangan
POST	/auth/register	Public	Registrasi akun baru
POST	/auth/login	Public	Login, mendapat access + refresh token
POST	/auth/refresh	Public	Perbarui access token
POST	/auth/logout	Protected	Blacklist/invalidasi refresh token
GET	/auth/profile	Protected	Data profil user yang sedang login
GET	/auth/google	Public	Redirect ke Google screen
GET	/auth/google/callback	Public	Callback setelah Google OAuth berhasil

4.2 Property Service (/Properties/*, /rooms/*, /owners/*)

Method	Path	Akses	Keterangan
GET	/properties	Public	List properti dengan filter & pagination
POST	/properties	Protected	Tambah properti baru
GET	/properties/:id	Public	Detail properti
PUT	/properties/:id	Protected	Update properti
DELETE	/properties/:id	Protected	Hapus properti

GET	/rooms	Public	List kamar dengan filter & pagination
POST	/rooms	Protected	Tambah kamar baru
GET	/rooms/:id	Public	Detail kamar (dikonsumsi booking-service)
PUT	/rooms/:id	Protected	Update kamar
DELETE	/rooms/:id	Protected	Hapus kamar
POST	/owners	Protected	Tambah data pemilik
GET	/owners/:id	Protected	Detail pemilik beserta properti
PUT	/owners/:id	Protected	Update data pemilik

4.3 Booking Service (/bookings/*, /payments/*)

Method	Path	Akses	Keterangan
POST	/bookings	Protected	Buat booking (cek availability ke property-service)
GET	/bookings	Protected	List booking milik user dengan pagination
GET	/bookings/:id	Protected	Detail booking
PATCH	/bookings/:id/status	Proteced	Update status booking (approved/rejected/cancelled)
POST	/payments	Protected	Catat pembayaran sewa
GET	/payments/history	Protected	Riwayat pembayaran user

BAB 5 – IMPLEMENTASI FITUR UTAMA

5.1 Autentikasi JWT

- Access Token: Expire 15 menit, berisi id, email, role user

- Refresh Token: Expire 7 hari, disimpan di tabel refresh_tokens
- Blacklist: Saat logout, refresh token diinvalidasi di database sehingga tidak bisa digunakan lagi
- Middleware: JWT validation diterapkan sebagai middleware Express/Laravel, bukan dicek manual per endpoint

5.2 Google OAuth 2.0

- Access Token: Expire 15 menit, berisi id, email, role user
- Refresh Token: Expire 7 hari, disimpan di tabel refresh_tokens
- Blacklist: Saat logout, refresh token diinvalidasi di database sehingga tidak bisa digunakan lagi
- Middleware: JWT validation diterapkan sebagai middleware Express/Laravel, bukan dicek manual per endpoint

5.3 Rate Limiting

- Library: Express-rate-limit
- Limit: 60 request per menit per IP address
- Diterapkan di level gateway sebelum request diteruskan ke service
- Response saat limit terlampaui: 429 Too Many Requests

5.4 Database Schema Property Service

owners: id, user_id, name, phone, bank_account

hasMany

properties: id, owner_id, name, address, city, description

hasMany

rooms: id, property_id, room_number, type, price_per_month, status

hasMany

facilities: id, room_id, name

5.5 Pagination dan Filtering

Endpoint GET /properties dan GET /rooms mendukung:

- page dan per_page untuk pagination
- city untuk filter berdasarkan kota
- status untuk filter berdasarkan ketersediaan kamar

BAB 6 – KESIMPULAN

6.1 Kesimpulan

Sistem manajemen kos berbasis microservice berhasil dibangun dengan 3 service independen (auth, property, booking) dan 1 API Gateway. Seluruh requirement UTS telah terpenuhi:

- Minimal 3 microservice independen + 1 API Gateway
- Setiap service memiliki database terpisah
- Property service menggunakan PHP Laravel 11 dengan pola MVC
- Minimal 4 tabel relasional di property service
- JWT authentication dengan refresh token dan blacklist logout
- Google OAuth 2.0 Authorization Code Flow
- Rate limiting 60 request/menit per IP di gateway
- HTTP method dan response code sesuai standar REST
- Pagination dan filtering pada endpoint listing
- Inter-service communication via HTTP (booking → property)